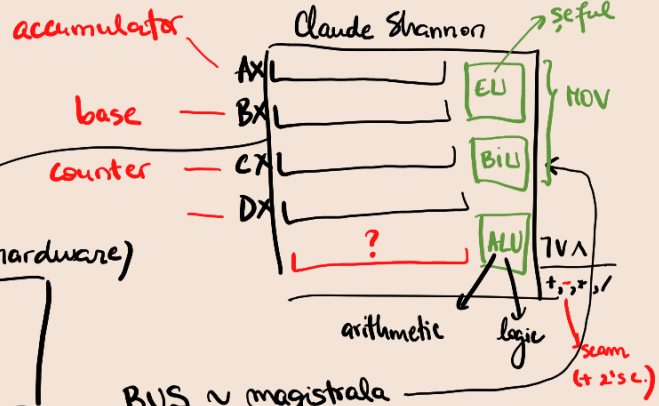
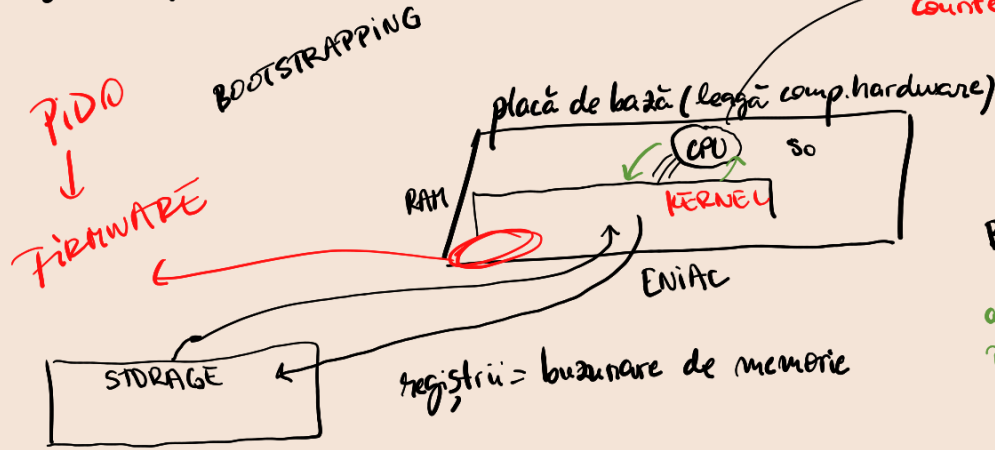


17.10.2025

Logica → Aristotel
Logica = Algebră → Boole



BUS ~ magistrala

↓

autostrada care face legătura între hardware, software

↓

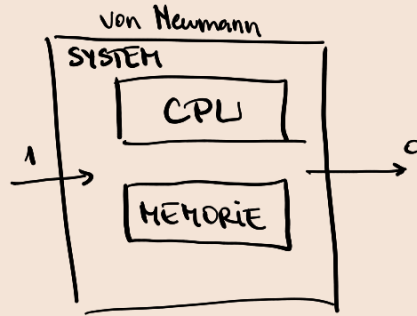
permite input, output => e o interfață

PROCEDURAL PROGRAMMING.

Von Neumann a folosit Divide et Impera pt. a sparge în buc. Machine Turing.

dominante

INTERFATA

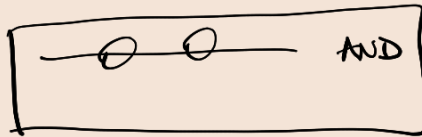
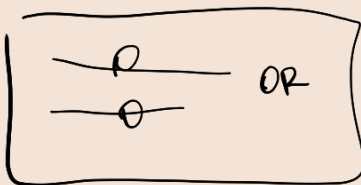


ENVIRONMENT

XOR → problematic

→ desfiintat problema perceptronului.

transistori YT : "How transistors run code?"



$$\int = \lim \sum \text{ (Riemann)}$$

$$2^0, 2^1, 2^2, \dots, 2^n \rightarrow 2^{n+1}$$

$$\frac{\partial f}{\partial x} \rightarrow \lim_{\Delta x \rightarrow 0} \frac{\partial f}{\Delta x} \rightarrow \frac{\partial f}{\partial x}$$

↓
derivata parțială.

Total se reduce la u^n .

⊗ → T când sunt ≠
⊗ → T când sunt toate T.

$$\begin{array}{c} 0+ \\ 0 \\ \hline 00 \\ \wedge \otimes \end{array} \quad \begin{array}{c} 0+ \\ 1 \\ \hline 01 \\ \wedge \otimes \end{array} \quad \begin{array}{c} 1+ \\ 0 \\ \hline 01 \\ \wedge \otimes \end{array} \quad \begin{array}{c} 1+ \\ 1 \\ \hline 10 \\ \wedge \otimes \end{array}$$

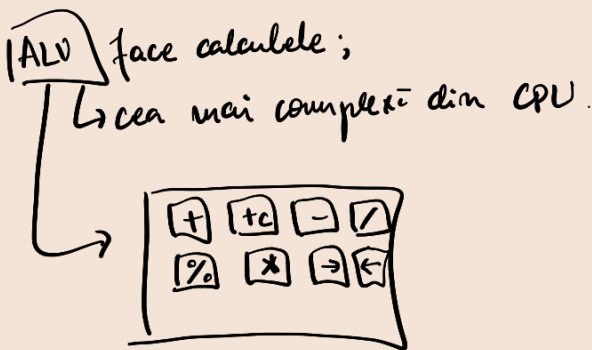
$$\begin{array}{l} \left[\begin{array}{l} 0 \otimes 0 = 0 \\ 0 \wedge 0 = 0 \end{array} \right] \quad \left[\begin{array}{l} 1 \otimes 0 = 1 \\ 1 \wedge 0 = 0 \end{array} \right] \\ \left[\begin{array}{l} 0 \otimes 1 = 1 \\ 0 \wedge 1 = 0 \end{array} \right] \quad \left[\begin{array}{l} 1 \otimes 1 = 0 \\ 1 \wedge 1 = 1 \end{array} \right] \end{array}$$

8 bit + 8 bit → 8 bit + 1.
8 bit · 8 bit → 16 bit

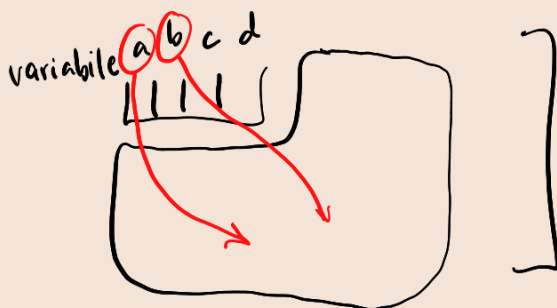
Cea mai semnificativ bit: stângă

DNF → De Morgan $\neg V \Rightarrow \neg \wedge$

$$(p \wedge \neg q) \vee (\neg p \wedge q) \equiv p \otimes q$$



code segment + data segment ⇒ PROGRAM (.com, .exe)



domnișoarele au separat
partea variabilă a unui
program de cea non-var.

STORED PROCEDURE

ASCII CODE

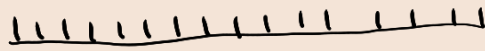
$$\{0,1\} \times \{0,1\} = \{(0,0), (0,1), (1,0), (1,1)\}$$

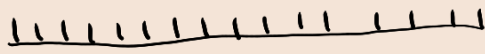
$$\{0,1\}^8 = \{\underbrace{0,0,0,\dots,0}_8\}, \quad \text{! ASCII}$$

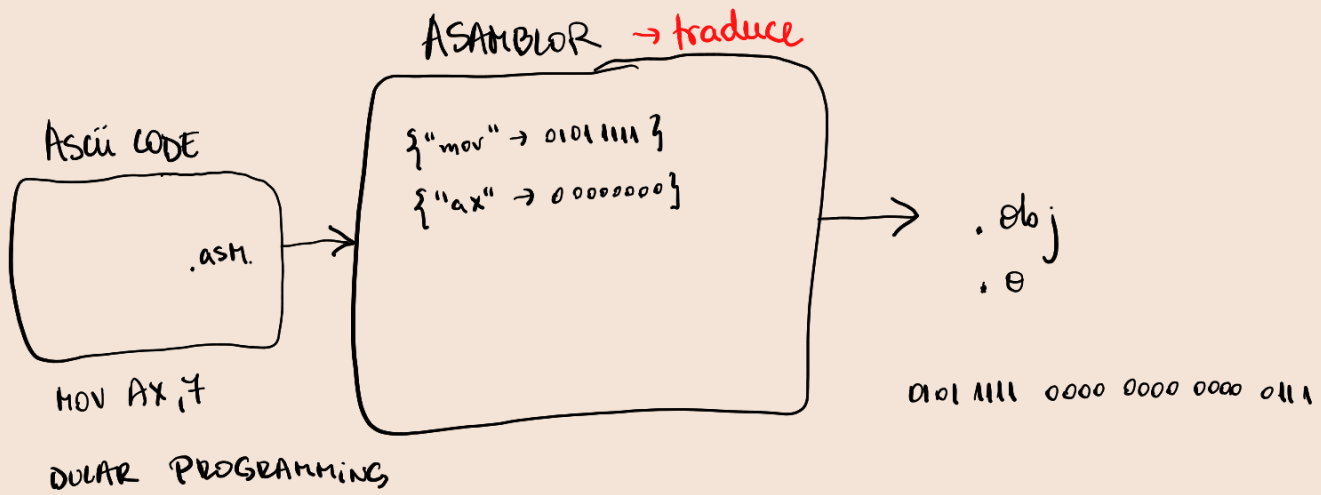
doua de def.
pt. un byte

$(0, 1, 0, 0, 0, 0, 0, 1) = 'a' \rightarrow \text{mov}$
 $(0, 1, 0, 0, 0, 0, 1, 0) = 'b' \rightarrow \text{adp}$
 $(1, 0, 1, 0, 1, 1, 1, 1) \rightarrow \text{Dw.}$
 $(1, \dots, 1) = 255$

```
MOV AX, a
MOV BX, b
ADD AX, BX → AX
```

AX: 

BX: 

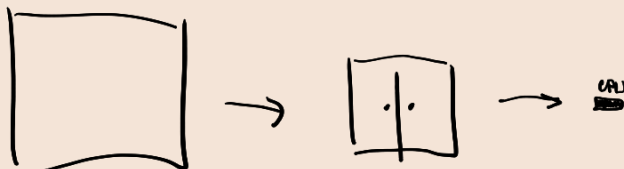


Primerul compilator: Grace Hopper.

Transistor → 

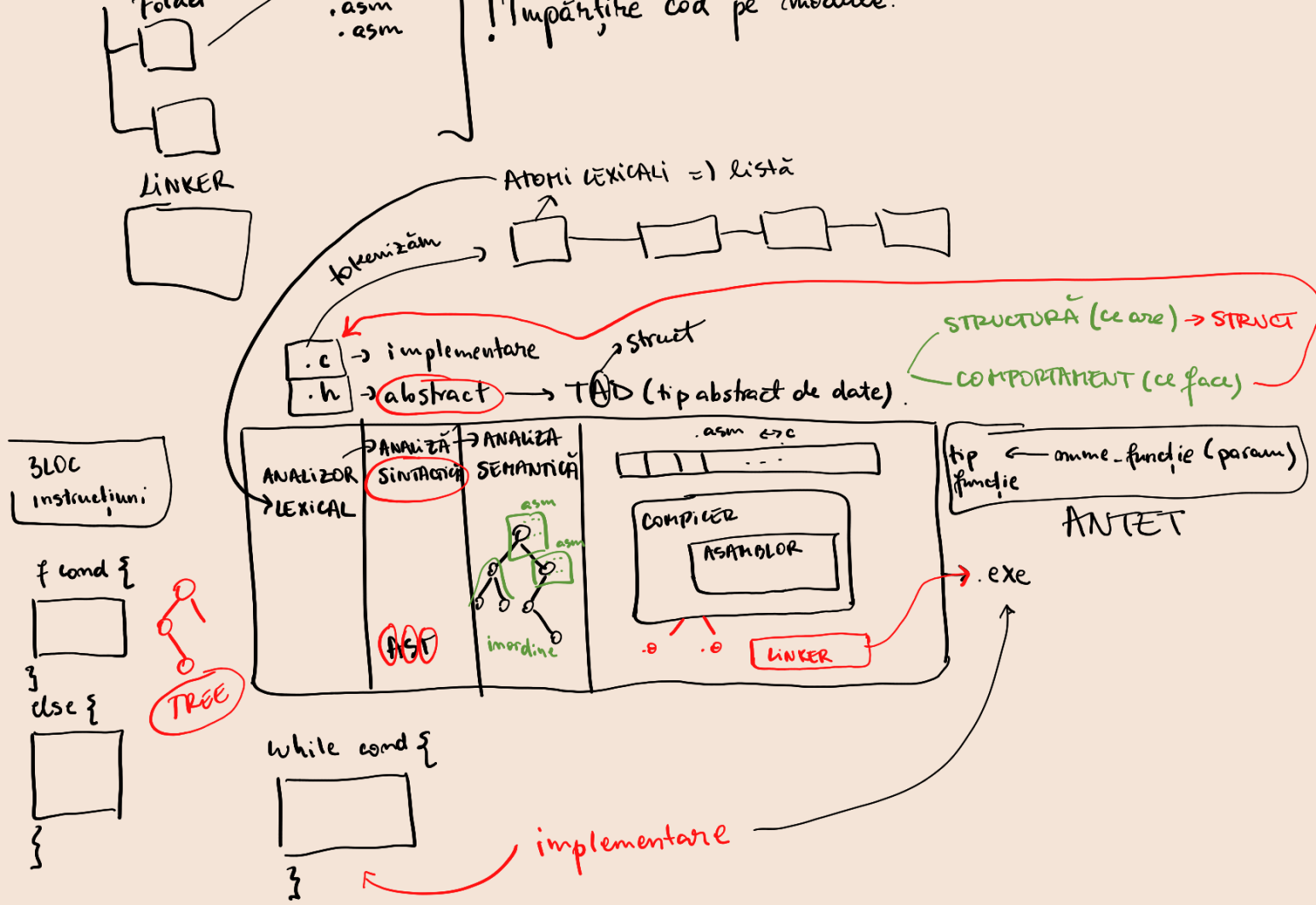
Shockley → a micșorat tranzistorul pt. misiunea Apollo 11?

ia mintile luminate de la univ. de top ↑ Boston, Yale, Princeton etc.
 cu ego a plecat din trio



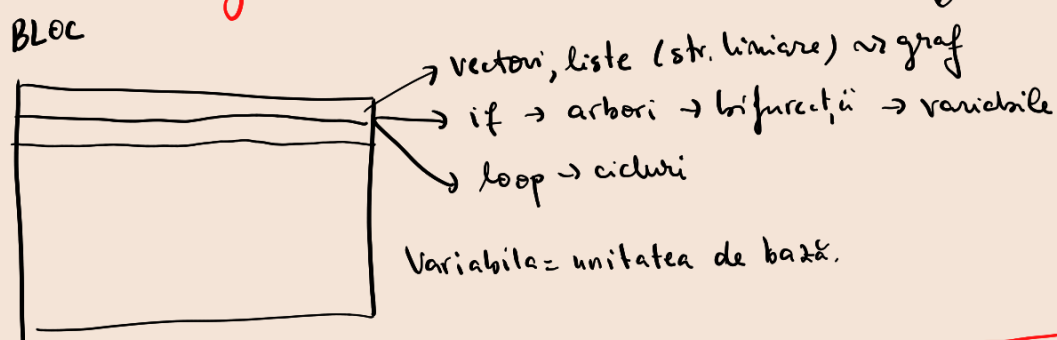
microcip 90% se consumă în calcule → cooler

module



3 tipuri de building-block-uri

BLOC



- int
 - char
 - float
 - short
 - ~~bool~~
- id 0
 - const 1
 - if 2
 - dox 3
 - while 4
 - stat 5
 - void 6
 - ?

int a = 7
in there?

Start: " (=) " main()

TDD -> FDD

in storage se încarcă tot în curent electric și trece prin RAM, acum devine volatilă, stă în coadă de prioritate, initial location of the CSIP.

/main in RAM

ps - All procesele in Unix

file system = repo de fișiere

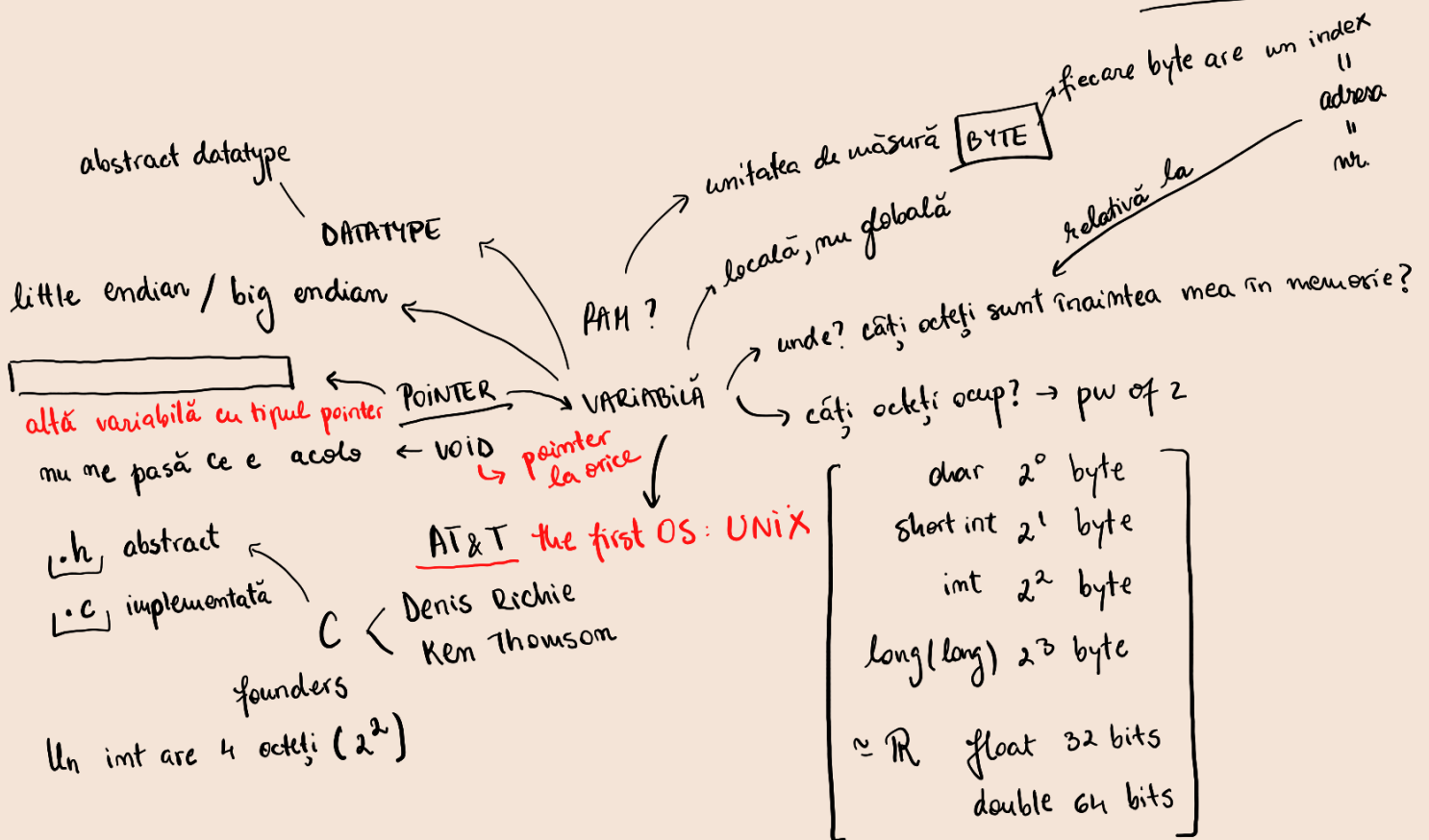
jido

offset 14h → CSIP → reg. processor. IP. (Instruction Pointer)

Pointer

↳ o zonă de memorie → val → m → adresa unei
alte zone de memorie
↳ are o adresă cu valoare

24.10.2025



POINTER: O zonă de memorie care conține în interiorul ei o variabilă care este interpretată la ^{altă} adresă de memorie.

- număr - labelul asigmat unei adrese
- facem o variabilă locală. NU GLOBALĂ.
- dăm denumiri SUGESTIVE

→ la printf mereu încheiem cu \n sau \o, altfel nu dă flash.

int* char* void* A* → au aceeași dimensiune

↓
depinde de procesor. X32
X64

! Pointer ARE adresa.

↳ are în el un nr care e de fapt o adresă)

! Totdeauna inițializăm variabilele.

int* val = &nr; (⇒) int *val = ...

↓
"pointer la int"

↓
și luăm adresa de unde se află în memorie

~~int a;~~

int main()

{

int numar = 247777216;

void* pointer_la_numar = &numar // → 0x7ffe6fb29174 în hexa

Char* pointer_la_octetul_0 = &numar

Char* pointer_la_octetul_1 = pointer_la_octetul_0 + 1

Char* pointer_la_octetul_2 = pointer_la_octetul_1 + 1

Char* pointer_la_octetul_3 = pointer_la_octetul_2 + 1

printf("%p\n", *pointer_la_numar)

printf("format: \"%p\n\", &numar)

*(pointer_la_octetul_1)++

*(pointer_la_octetul_3)--

tipul de
dată
"pointer
la
void"



} return 0;

adresa 0x7f580

→ înseamnă orice
* void
↓
ia oricât

Explicit: mergi la adresă
↓
cât e tipul pointerului?

* mr

dereferențiere a pointerului mr.

↓
vede adresa

↓
ia de acolo 4 octeți (2² byt)
int.

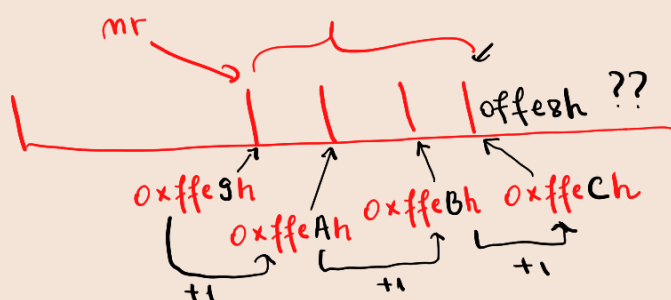
interpretază

int mr = 7 (valoare)

int * pointer_la_mr = 8 mr → "se schimbă tipul de dată"

char * pointer_la_octetul_0 = 8 mr

char * pointer_la_octetul_1 = pointer_la_octetul_0 + 1



adresa + câți-octeți-are-un int } crește valoarea
+4

adresa + 1 } crește adresa

Pointer la int, → ie 4 octeți ⇒ convert la cu mai mic
7 ocupă de fel de mult
Pointer la char
→ ie 1 octet

→ one decim in int case e little endian

11111111 00000000 00000000 00001111
octet 3

→ onean 7 0 0 0 → pointer la char

octet 0 octet 1 octet 2 octet 3

↓
dereferentiere

char * pointer la octetul 1

||
char

↓
adresa la acest octet

adresa

↓
octet

unsigned char *